

SAMFormer-APCP

hx2504

December 2025

1 Model Architecture

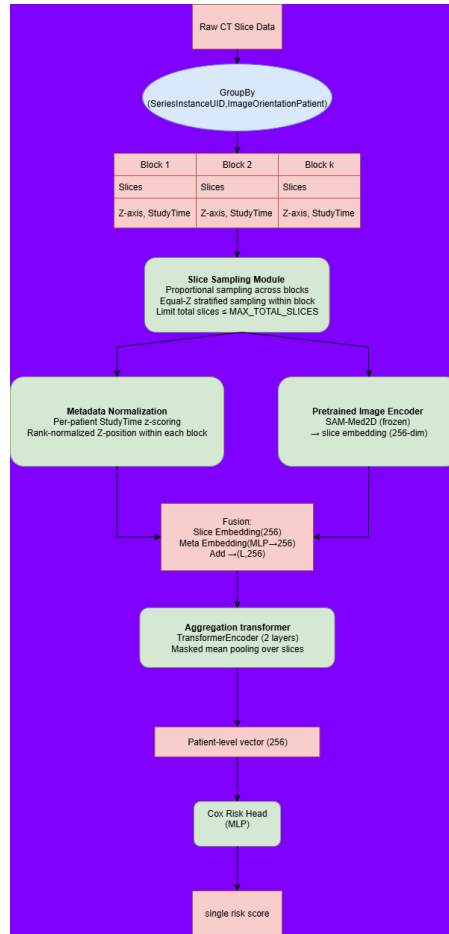
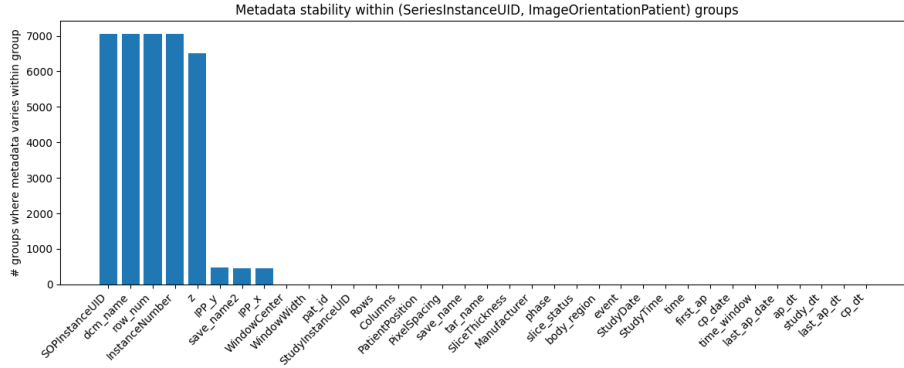


Figure 1: architecture

2 CT block and Sampling

To sample representatively from the hundreds/thousands of slices per patient, we first partition each patient’s scans into homogeneous blocks, where slices share as much spatial and temporal metadata as possible. As shown in the



figure, after excluding identity-like fields, slices within each block (grouped by SeriesInstanceUID and ImageOrientationPatient) differ primarily in Z position, while remaining constant in other geometric/spatial/time-related metadata. Rare deviations exist, but they are infrequent and acceptable for our sampling purpose.

2.1 sampler(representativeness and denoise)

Therefore, given a maximum slice budget per patient, we first allocate a per-block quota proportionally to each group’s slice count. Within each block, we then bin slices along the Z-axis into the assigned number of bins with equal Z-intervals. During training, we randomly sample one slice per bin. For validation and testing, we instead select the center slice of each bin deterministically to ensure reproducibility.

In addition, to reduce noise from unrelated anatomy, we bias the sampler toward slices explicitly labeled as pancreas: within each Z-bin, we sample only from pancreas-labeled slices (randomly during training, and by selecting the bin-center slice for validation/testing).

3 Postion encoding and image encoding

3.1

The image encoder of a pretrained **SAM-Med2D** model serves as a strong medical-image feature extractor. After windowing and normalization (using window center and window width), each slice is mapped to a [256, 16, 16] feature

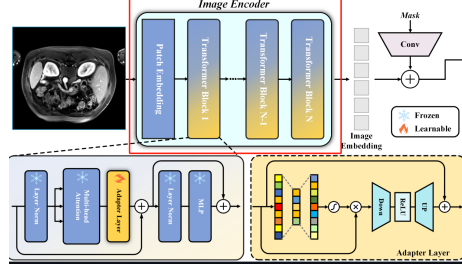


Figure 2: Feature Extractor

map. We then apply mean pooling over the last two spatial dimensions to obtain a 256-dimensional slice representation.

3.2

When feeding slice embeddings into the transformer, we use spatial- and time-related metadata as positional encodings. Specifically, we normalize the **StudyTime** (which varies across blocks) and the **Z-position** (which varies across slices), and concatenate the two scalars.

Normalization of spatial and temporal metadata. To construct positional encodings, we normalize **Z-position** and study time on different scopes to make them comparable and stable.

Z-position (within-block rank normalization). Within each block (grouped by **SeriesInstanceUID** and **ImageOrientationPatient**), we compute an orientation-aware slice coordinate by projecting the slice origin **ImagePositionPatient** onto the slice normal vector derived from **ImageOrientationPatient**:

$$s_i = \langle \mathbf{n}, \mathbf{p}_i \rangle, \quad \mathbf{n} = \mathbf{r} \times \mathbf{c},$$

where $\mathbf{r}$ and $\mathbf{c}$ are the row/column direction cosines from **ImageOrientationPatient**, and $\mathbf{p}_i$ is the 3D position from **ImagePositionPatient**. If these fields are unavailable, we fall back to a provided scalar $\mathbf{z}$. We then convert $\{s_i\}$ into a *rank-normalized* coordinate in $[0, 1]$:

$$\tilde{z}_i = \frac{\text{rank}(s_i)}{N - 1},$$

where N is the number of slices in the block. If $N = 1$, we set $\tilde{z}_i = 0$.

Study time (within-patient z-score normalization). For the temporal feature, we normalize **StudyTime** at the patient level using z-score normalization computed over all valid (times that are finite) studies for that patient:

$$\tilde{t}_i = \frac{t_i - \mu_t}{\sigma_t},$$

where μ_t and σ_t are the mean and standard deviation of valid t_i values for the patient. To ensure numerical stability, if σ_t is non-finite or smaller than a threshold (e.g., 10^{-6}), we set $\sigma_t = 1$. Any resulting NaNs are replaced by 0.

4 Transformer-based Slice Aggregator

Given a variable-length sequence of slice embeddings $\mathbf{x} \in R^{L \times D}$ produced by the pretrained SAM-Med2D image encoder, we aggregate slice-level information into a single patient representation using a Transformer encoder.

Metadata-conditioned positional encoding. For each slice, we also construct a metadata vector $\mathbf{m} \in R^{L \times 2}$ containing normalized spatial/temporal attributes (e.g., normalized study time and within-block rank-normalized z-position). Instead of using fixed sinusoidal encodings, we map metadata to a learnable positional embedding through a lightweight MLP:

$$\mathbf{p} = f_{MLP}(\mathbf{m}), \quad \mathbf{p} \in R^{L \times D}.$$

The positional embedding is then added to the slice embedding:

$$\tilde{\mathbf{x}} = \mathbf{x} + \mathbf{p}.$$

Transformer encoder for inter-slice modeling. We pass $\tilde{\mathbf{x}}$ through a N -layer Transformer encoder with H attention heads to model dependencies and interactions across slices:

$$\mathbf{h} = \text{TransformerEncoder}(\tilde{\mathbf{x}}),$$

while using a padding mask to ignore padded positions for patients with fewer than L slices.

Patient-level pooling. Finally, we obtain a fixed-dimensional patient representation by masked mean pooling over the slice dimension:

$$\mathbf{v} = \frac{\sum_{l=1}^L \mathbf{h}_{:,l,:} \mathbf{1}[\text{valid}_l]}{\sum_{l=1}^L \mathbf{1}[\text{valid}_l] + \epsilon}, \quad \mathbf{v} \in R^D.$$

A final linear projection is applied to produce the aggregated patient embedding used by the downstream risk prediction head.

5 Training Setup

We train the patient-level risk model using a Cox partial-likelihood objective with gradient accumulation. Each minibatch contains $B = 16$ patients, and we accumulate gradients for 4 steps, yielding an effective batch size of 64. This is

important because Cox loss can be unstable with very small batches: when the batch is tiny, the within-batch risk sets are small and the resulting gradients become noisy. Gradient accumulation provides a more stable optimization signal without exceeding GPU memory.

Data loading. We construct train/validation/test datasets at the patient level and use a dynamic padding collate function to handle variable-length slice sequences. Training data are shuffled, while validation and test are evaluated deterministically.

Backbone freezing. We load a pretrained SAM-Med2D image encoder (ViT-B) as the slice feature extractor and freeze all its parameters to reduce memory usage and improve training stability. Only the downstream aggregation and risk head (either cox or deepsurv) are optimized.

Optimization and scheduler. We optimize the trainable parameters using Adam with weight decay. The learning rate follows a warmup + cosine schedule over 100 epochs.

Model selection and evaluation. We monitor fixed-horizon AUC at 5 and 10 years on the validation set each epoch, and select the best checkpoint based on the 5-year AUC.

Slice budget and sampling trade-off. The maximum number of sampled slices per patient is tuned to balance two competing effects: (i) sampling more slices improves coverage but dilutes pancreas-related signal with unrelated anatomy, whereas (ii) sampling fewer slices increases pancreas proportion but can introduce higher variance and sensitivity to sampling noise. It turns out 64 slices is the optimal choice